

ASSIGNMENT NO: 2

NETWORK LAB EXPERIMENTS

Name	Muhammed Aamir
Class	S5, CSE A
Roll No.	48
Date	08/08/2026

Question 1: Hello – Client/Server Communication

Aim

To create a simple TCP socket-based client and server in C, where the client establishes a connection, transmits a greeting, and receives a response from the server.

Commands Used

```
gcc server.c -o server
```

```
gcc client.c -o client
```

```
./server (run in Terminal 1)
```

```
./client (run in Terminal 2)
```

Procedure

The server creates a TCP socket, binds it to port 8080, listens for an incoming connection, and accepts the client. The client connects to localhost and sends a greeting. After receiving the message, the server sends a reply back to the client.

Client Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080
#define BUFFER_SIZE 1024
#define SERVER_IP "127.0.0.1"

int main() {
    int client_fd;
    struct sockaddr_in server_addr;
    char buffer[BUFFER_SIZE] = {0};
    const char *message = "Hello Server";

    client_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (client_fd < 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    if (inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr) <= 0) {
        perror("invalid address");
        close(client_fd);
        exit(EXIT_FAILURE);
    }

    if (connect(client_fd, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
        perror("connect failed");
        close(client_fd);
        exit(EXIT_FAILURE);
    }

    printf("Connected to server %s:%d\n", SERVER_IP, PORT);
    send(client_fd, message, strlen(message), 0);
    printf("Sent to server: %s\n", message);

    ssize_t bytes_received = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
    if (bytes_received > 0) {
        buffer[bytes_received] = '\0';
        printf("Received from server: %s\n", buffer);
    }
}
```

```

    }

    close(client_fd);
    return 0;
}

```

Server Code

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t client_len = sizeof(client_addr);
    char buffer[BUFFER_SIZE] = {0};
    const char *reply = "Hello Client";

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd < 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
        perror("bind failed");
        close(server_fd);
        exit(EXIT_FAILURE);
    }

    if (listen(server_fd, 3) < 0) {
        perror("listen failed");
        close(server_fd);
        exit(EXIT_FAILURE);
    }

    printf("Server listening on port %d...\n", PORT);
    client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &client_len);
    if (client_fd < 0) {
        perror("accept failed");
        close(server_fd);
        exit(EXIT_FAILURE);
    }

    printf("Client connected: %s:%d\n",
           inet_ntoa(client_addr.sin_addr),
           ntohs(client_addr.sin_port));

    ssize_t bytes_received = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
    if (bytes_received > 0) {
        buffer[bytes_received] = '\0';
        printf("Received from client: %s\n", buffer);
    }

    send(client_fd, reply, strlen(reply), 0);
    printf("Sent to client: %s\n", reply);
}

```

```
    close(client_fd);  
    close(server_fd);  
    return 0;  
}
```

```
aamir@AamirsPC: /mnt/c/Use  X + v  
Setting up g++-x86-64-linux-gnu (4:15.2.0-5ubuntu1) ...  
Setting up g++-15 (15.2.0-16ubuntu1) ...  
Setting up g++ (4:15.2.0-5ubuntu1) ...  
update-alternatives: using /usr/bin/g++ to provide /usr/bin/c++ (c++) in auto mode  
Setting up build-essential (12.12ubuntu2.26.04.2) ...  
Processing triggers for systemd (259.5-0ubuntu3) ...  
Processing triggers for man-db (2.13.1-1build1) ...  
Processing triggers for libc-bin (2.43-2ubuntu2.3) ...  
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab$ gcc server.c -o server  
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab$ ls  
client.c  server  server.c  
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab$ ./server  
Server listening on port 8080...  
Client connected: 127.0.0.1:46194  
Received from client: Hello Server  
Sent to client: Hello Client  
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab$
```

```
aamir@AamirsPC: /mnt/c/Use  X + v  
aamir@AamirsPC:~$ cd "/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab"  
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab$ ./client  
-bash: ./client: No such file or directory  
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab$ gcc client.c -o client  
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab$ ./client  
Connected to server 127.0.0.1:8080  
Sent to server: Hello Server  
Received from server: Hello Client  
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab$ |
```

Conclusion

The TCP client and server communicated successfully through localhost. The greeting sent by the client was received by the server, which returned a confirmation message, demonstrating two-way socket communication.

Question 2: Arithmetic Server

Aim

To develop a TCP client-server program where the client provides two integer values and the server performs the basic arithmetic operations and sends the calculated results back.

Commands Used

```
gcc server.c -o server
gcc client.c -o client
./server (run in Terminal 1)
./client (run in Terminal 2)
```

Procedure

The client accepts two integers and transmits them to the server. The server extracts the values, calculates their sum, difference, product, and quotient, and then sends the formatted results to the client. Division by zero is handled separately.

Client Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8081
#define BUFFER_SIZE 1024
#define SERVER_IP "127.0.0.1"

int main() {
    int client_fd;
    struct sockaddr_in server_addr;
    char buffer[BUFFER_SIZE] = {0};
    char message[BUFFER_SIZE] = {0};
    int a, b;

    client_fd = socket(AF_INET, SOCK_STREAM, 0);
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr);
    connect(client_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));

    printf("Connected to server %s:%d\n", SERVER_IP, PORT);
    printf("Enter first integer: ");
    scanf("%d", &a);
    printf("Enter second integer: ");
    scanf("%d", &b);

    snprintf(message, BUFFER_SIZE, "%d %d", a, b);
    send(client_fd, message, strlen(message), 0);
    printf("Sent to server: %s\n", message);

    int n = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
    buffer[n] = '\0';
    printf("Results received from server:\n%s\n", buffer);

    close(client_fd);
    return 0;
}
```

Server Code

```
#include <stdio.h>
```

```

#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8081
#define BUFFER_SIZE 1024

int main() {
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t client_len = sizeof(client_addr);
    char buffer[BUFFER_SIZE] = {0};
    char result[BUFFER_SIZE] = {0};

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);
    bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
    listen(server_fd, 3);

    printf("Server listening on port %d...\n", PORT);
    client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &client_len);
    printf("Client connected: %s:%d\n",
        inet_ntoa(client_addr.sin_addr), ntohs(client_addr.sin_port));

    int n = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
    buffer[n] = '\0';
    printf("Received from client: %s\n", buffer);

    int a, b;
    sscanf(buffer, "%d %d", &a, &b);

    int sum = a + b;
    int diff = a - b;
    int prod = a * b;
    if (b != 0) {
        double quo = (double)a / b;
        snprintf(result, BUFFER_SIZE,
            "Sum = %d\nDifference = %d\nProduct = %d\nQuotient = %.2f",
            sum, diff, prod, quo);
    } else {
        snprintf(result, BUFFER_SIZE,
            "Sum = %d\nDifference = %d\nProduct = %d\nQuotient = Undefined (division by zero)",
            sum, diff, prod);
    }

    send(client_fd, result, strlen(result), 0);
    printf("Sent results to client:\n%s\n", result);

    close(client_fd);
    close(server_fd);
    return 0;
}

```

```
aamir@AamirsPC: /mnt/c/Use x + v
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab$ cd q2
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q2$ gcc server.c -o server
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q2$ ./server
Server listening on port 8081...
Client connected: 127.0.0.1:59538
Received from client: 15 4
Sent results to client:
Sum = 19
Difference = 11
Product = 60
Quotient = 3.75
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q2$
```

```
aamir@AamirsPC: /mnt/c/Use x + v
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q2$ gcc client.c -o client
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q2$ ./client
Connected to server 127.0.0.1:8081
Enter first integer: 15
Enter second integer: 4
Sent to server: 15 4
Results received from server:
Sum = 19
Difference = 11
Product = 60
Quotient = 3.75
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q2$
```

Conclusion

The arithmetic client-server application successfully transferred two integers and returned the required calculations. For the demonstrated input 15 and 4, the expected results are 19, 11, 60, and 3.75 respectively.

Question 3: String Processing Server

Aim

To implement a TCP-based string processing service in which the server receives text from the client, finds its length, changes it to uppercase, reverses it, and returns all three results.

Commands Used

```
gcc server.c -o server
```

```
gcc client.c -o client
```

```
./server (run in Terminal 1)
```

```
./client (run in Terminal 2)
```

Procedure

The client reads a line of text and sends it to the server. The server determines the string length, generates an uppercase version, creates a reversed copy, and packages the results into a single response.

Client Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8082
#define BUFFER_SIZE 1024
#define SERVER_IP "127.0.0.1"

int main() {
    int client_fd;
    struct sockaddr_in server_addr;
    char buffer[BUFFER_SIZE] = {0};
    char input[BUFFER_SIZE];

    client_fd = socket(AF_INET, SOCK_STREAM, 0);
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr);
    connect(client_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));

    printf("Connected to server %s:%d\n", SERVER_IP, PORT);
    printf("Enter a string: ");
    fgets(input, BUFFER_SIZE, stdin);
    input[strcspn(input, "\n")] = '\0'; // remove trailing newline

    send(client_fd, input, strlen(input), 0);
    printf("Sent to server: %s\n", input);

    int n = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
    buffer[n] = '\0';
    printf("Results received from server:\n%s\n", buffer);

    close(client_fd);
    return 0;
}
```

Server Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <arpa/inet.h>
```

```

#include <sys/socket.h>

#define PORT 8082
#define BUFFER_SIZE 1024

int main() {
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t client_len = sizeof(client_addr);
    char buffer[BUFFER_SIZE] = {0};
    char upper[BUFFER_SIZE] = {0};
    char reversed[BUFFER_SIZE] = {0};
    char result[BUFFER_SIZE * 3] = {0};

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);
    bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
    listen(server_fd, 3);

    printf("Server listening on port %d...\n", PORT);
    client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &client_len);
    printf("Client connected: %s:%d\n",
        inet_ntoa(client_addr.sin_addr),
        ntohs(client_addr.sin_port));

    int n = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
    buffer[n] = '\0';
    printf("Received from client: %s\n", buffer);

    int len = strlen(buffer);
    for (int i = 0; i < len; i++)
        upper[i] = toupper(buffer[i]);
    upper[len] = '\0';

    for (int i = 0; i < len; i++)
        reversed[i] = buffer[len - 1 - i];
    reversed[len] = '\0';

    snprintf(result, sizeof(result),
        "Length = %d\nUppercase = %s\nReversed = %s",
        len, upper, reversed);

    send(client_fd, result, strlen(result), 0);
    printf("Sent results to client:\n%s\n", result);

    close(client_fd);
    close(server_fd);
    return 0;
}

```

```
aamir@AamirsPC: /mnt/c/Use x + v
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q2$ cd "/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q3"
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q3$ gcc server.c -o server
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q3$ ./server
Server listening on port 8082...
Client connected: 127.0.0.1:38228
Received from client: dacarys vermax
Sent results to client:
Length = 14
Uppercase = DACARYS VERMAX
Reversed = xamrev syracad
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q3$

aamir@AamirsPC: /mnt/c/Use x + v
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q3$ gcc client.c -o client
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q3$ ./client
Connected to server 127.0.0.1:8082
Enter a string: dacarys vermax
Sent to server: dacarys vermax
Results received from server:
Length = 14
Uppercase = DACARYS VERMAX
Reversed = xamrev syracad
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q3$
```

Conclusion

The server correctly processed the supplied text and returned its length, uppercase representation, and reverse order. This confirms successful transmission and string manipulation using TCP sockets.

Question 4: Array Processing Server

Aim

To build a client-server application that accepts an integer array from the client and calculates the total, arithmetic average, largest value, and smallest value at the server.

Commands Used

```
gcc server.c -o server
```

```
gcc client.c -o client
```

```
./server (run in Terminal 1)
```

```
./client (run in Terminal 2)
```

Procedure

The client first sends the number of elements and then the array values. The server reconstructs the array from the received message, processes each element, and sends the calculated statistics to the client.

Client Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8083
#define BUFFER_SIZE 1024
#define SERVER_IP "127.0.0.1"

int main() {
    int client_fd;
    struct sockaddr_in server_addr;
    char buffer[BUFFER_SIZE] = {0};
    char message[BUFFER_SIZE] = {0};
    int arr[100], N;

    client_fd = socket(AF_INET, SOCK_STREAM, 0);
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr);
    connect(client_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));

    printf("Connected to server %s:%d\n", SERVER_IP, PORT);
    printf("Enter number of elements (N): ");
    scanf("%d", &N);
    printf("Enter %d integers:\n", N);

    char temp[16];
    snprintf(message, BUFFER_SIZE, "%d", N);
    for (int i = 0; i < N; i++) {
        scanf("%d", &arr[i]);
        snprintf(temp, sizeof(temp), " %d", arr[i]);
        strcat(message, temp);
    }

    send(client_fd, message, strlen(message), 0);
    printf("Sent to server: %s\n", message);

    int n = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
    buffer[n] = '\0';
    printf("Results received from server:\n%s\n", buffer);

    close(client_fd);
    return 0;
}
```

Server Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8083
#define BUFFER_SIZE 1024

int main() {
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t client_len = sizeof(client_addr);
    char buffer[BUFFER_SIZE] = {0};
    char result[BUFFER_SIZE] = {0};

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);
    bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
    listen(server_fd, 3);

    printf("Server listening on port %d...\n", PORT);
    client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &client_len);
    printf("Client connected: %s:%d\n",
        inet_ntoa(client_addr.sin_addr), ntohs(client_addr.sin_port));

    int n = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
    buffer[n] = '\0';
    printf("Received from client: %s\n", buffer);

    int arr[100], count = 0;
    char *token = strtok(buffer, " ");
    int N = atoi(token);
    token = strtok(NULL, " ");
    while (token != NULL && count < N) {
        arr[count++] = atoi(token);
        token = strtok(NULL, " ");
    }

    int sum = 0, max = arr[0], min = arr[0];
    for (int i = 0; i < count; i++) {
        sum += arr[i];
        if (arr[i] > max) max = arr[i];
        if (arr[i] < min) min = arr[i];
    }

    double avg = (double)sum / count;

    snprintf(result, BUFFER_SIZE,
        "Sum = %d\nAverage = %.2f\nMaximum = %d\nMinimum = %d",
        sum, avg, max, min);

    send(client_fd, result, strlen(result), 0);
    printf("Sent results to client:\n%s\n", result);

    close(client_fd);
    close(server_fd);
    return 0;
}
```

```
aamir@AamirsPC: /mnt/c/Use x + v
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q3$ cd "/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q4"
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q4$ gcc server.c -o server
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q4$ ./server
Server listening on port 8083...
Client connected: 127.0.0.1:52154
Received from client: 5 12 7 25 3 18
Sent results to client:
Sum = 65
Average = 13.00
Maximum = 25
Minimum = 3
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q4$

aamir@AamirsPC: /mnt/c/Use x + v
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q3$ cd "/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q4"
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q4$ gcc client.c -o client
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q4$ ./client
Connected to server 127.0.0.1:8083
Enter number of elements (N): 5
Enter 5 integers:
12
7
25
3
18
Sent to server: 5 12 7 25 3 18
Results received from server:
Sum = 65
Average = 13.00
Maximum = 25
Minimum = 3
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q4$ |
```

Conclusion

The array server successfully received and processed the integer sequence. For the sample array {12, 7, 25, 3, 18}, it produces a sum of 65, an average of 13.00, a maximum of 25, and a minimum of 3.

Question 5: Matrix Analyzer

Aim

To design a TCP client-server program that accepts a square matrix, identifies whether it is diagonal, upper triangular, lower triangular, or general, and calculates its element sum and trace.

Commands Used

```
gcc server.c -o server
```

```
gcc client.c -o client
```

```
./server (run in Terminal 1)
```

```
./client (run in Terminal 2)
```

Procedure

The client reads the matrix order and all matrix elements, then sends them to the server. The server reconstructs the matrix, checks the required zero positions to classify its type, calculates the total of all elements, and finds the diagonal trace.

Client Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8084
#define BUFFER_SIZE 4096
#define SERVER_IP "127.0.0.1"
#define MAXN 20

int main() {
    int client_fd;
    struct sockaddr_in server_addr;
    char buffer[BUFFER_SIZE] = {0};
    char message[BUFFER_SIZE] = {0};
    int mat[MAXN][MAXN], N;

    client_fd = socket(AF_INET, SOCK_STREAM, 0);
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr);
    connect(client_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));

    printf("Connected to server %s:%d\n", SERVER_IP, PORT);
    printf("Enter order of square matrix (N): ");
    scanf("%d", &N);
    printf("Enter %d x %d matrix elements:\n", N, N);

    char temp[16];
    snprintf(message, BUFFER_SIZE, "%d", N);
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            scanf("%d", &mat[i][j]);
            snprintf(temp, sizeof(temp), " %d", mat[i][j]);
            strcat(message, temp);
        }
    }

    send(client_fd, message, strlen(message), 0);
    printf("Sent to server: %s\n", message);

    int n = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
    buffer[n] = '\0';
```

```

    printf("Results received from server:\n%s\n", buffer);

    close(client_fd);
    return 0;
}

```

Server Code

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8084
#define BUFFER_SIZE 4096
#define MAXN 20

int main() {
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t client_len = sizeof(client_addr);
    char buffer[BUFFER_SIZE] = {0};
    char result[BUFFER_SIZE] = {0};
    int mat[MAXN][MAXN];

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);
    bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
    listen(server_fd, 3);

    printf("Server listening on port %d...\n", PORT);
    client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &client_len);
    printf("Client connected: %s:%d\n",
        inet_ntoa(client_addr.sin_addr),
        ntohs(client_addr.sin_port));

    int n = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
    buffer[n] = '\0';
    printf("Received from client: %s\n", buffer);

    char *token = strtok(buffer, " ");
    int N = atoi(token);
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            token = strtok(NULL, " ");
            mat[i][j] = atoi(token);
        }
    }

    int isDiagonal = 1, isUpper = 1, isLower = 1;
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (i != j && mat[i][j] != 0) isDiagonal = 0;
            if (i > j && mat[i][j] != 0) isUpper = 0;
            if (i < j && mat[i][j] != 0) isLower = 0;
        }
    }

    char type[32];
    if (isDiagonal)
        strcpy(type, "Diagonal Matrix");
    else if (isUpper)
        strcpy(type, "Upper Triangular Matrix");
}

```

```

else if (isLower)
    strcpy(type, "Lower Triangular Matrix");
else
    strcpy(type, "General Matrix");

int sum = 0, trace = 0;
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        sum += mat[i][j];
    }
    trace += mat[i][i];
}

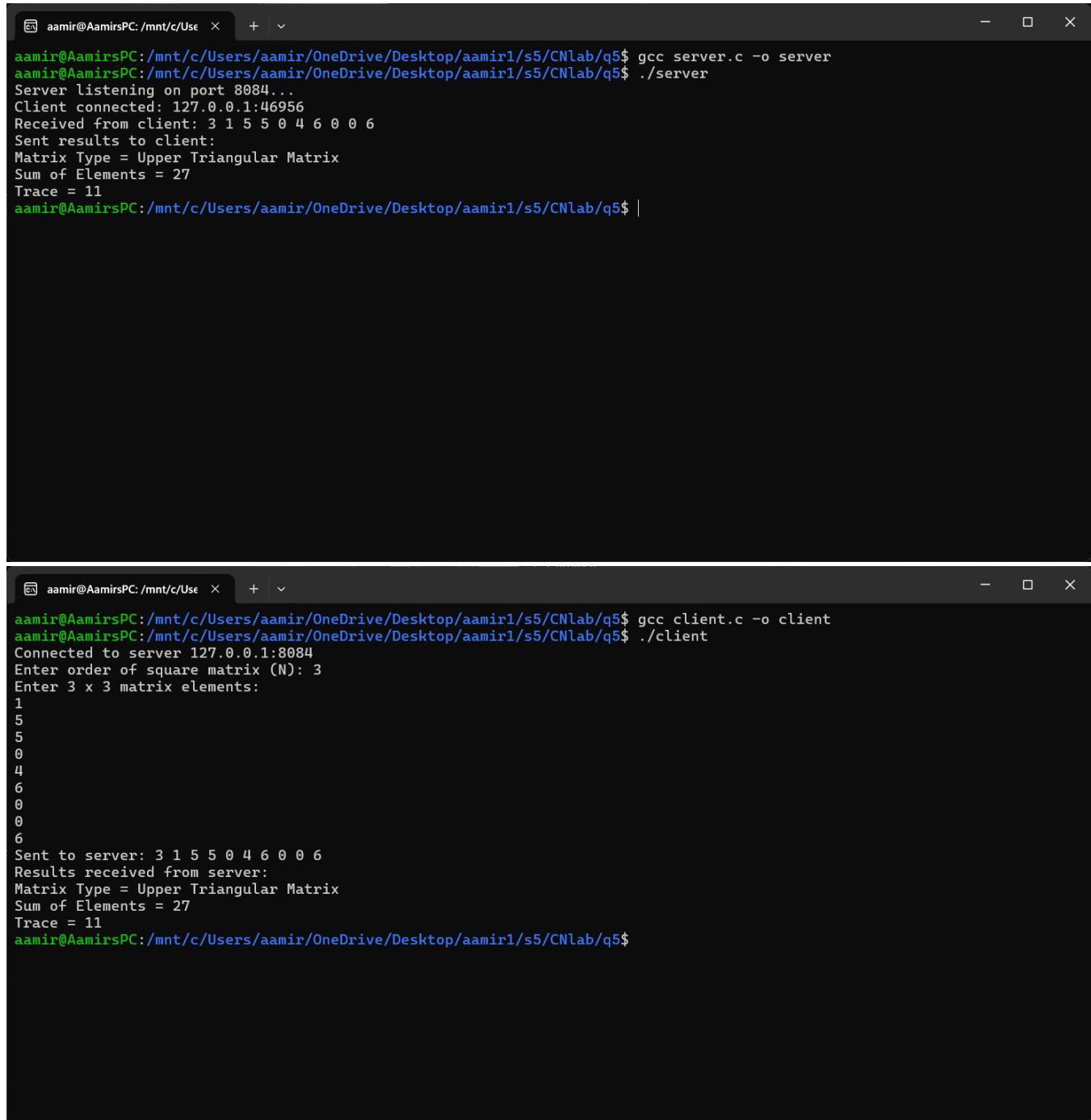
snprintf(result, BUFFER_SIZE,
    "Matrix Type = %s\nSum of Elements = %d\nTrace = %d",
    type, sum, trace);

send(client_fd, result, strlen(result), 0);
printf("Sent results to client:\n%s\n", result);

close(client_fd);
close(server_fd);
return 0;
}

```

Output Image Space



```
aamir@AamirsPC: /mnt/c/Use × + ▾
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q5$ gcc server.c -o server
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q5$ ./server
Server listening on port 8084...
Client connected: 127.0.0.1:46956
Received from client: 3 1 5 5 0 4 6 0 0 6
Sent results to client:
Matrix Type = Upper Triangular Matrix
Sum of Elements = 27
Trace = 11
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q5$ |

aamir@AamirsPC: /mnt/c/Use × + ▾
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q5$ gcc client.c -o client
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q5$ ./client
Connected to server 127.0.0.1:8084
Enter order of square matrix (N): 3
Enter 3 x 3 matrix elements:
1
5
5
0
4
6
0
0
6
Sent to server: 3 1 5 5 0 4 6 0 0 6
Results received from server:
Matrix Type = Upper Triangular Matrix
Sum of Elements = 27
Trace = 11
aamir@AamirsPC:/mnt/c/Users/aamir/OneDrive/Desktop/aamir1/s5/CNlab/q5$
```

Conclusion

The matrix analyzer successfully determines the matrix category and computes the sum and trace. For the sample 3×3 upper triangular matrix, the resulting type is Upper Triangular Matrix, with a total sum of 21 and a trace of 11.

— End of Assignment —